

Chapter 3

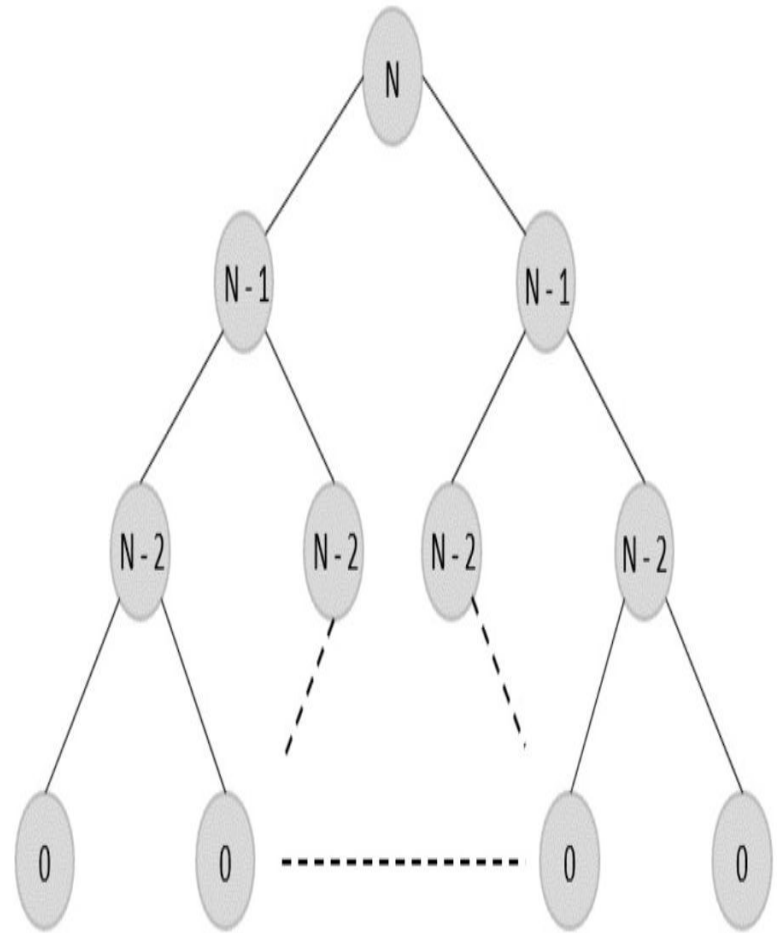
Dynamic Programming

Dynamic Programming

- Dynamic programming approach is similar to divide and conquer in breaking down the problem into smaller and yet smaller possible sub-problems.
- But unlike divide and conquer, these sub-problems are not solved independently. Rather, results of these smaller sub-problems are remembered and used for similar or overlapping sub-problems.
- Mostly, dynamic programming algorithms are used for solving optimization problems.
- Before solving the in-hand sub-problem, dynamic algorithm will try to examine the results of the previously solved sub-problems.
- The solutions of sub-problems are combined in order to achieve the best optimal final solution.
- This paradigm is thus said to be using Bottom-up approach.

So we can conclude that –

- The problem should be able to be divided into smaller overlapping sub-problem.
- Final optimum solution can be achieved by using an optimum solution of smaller sub-problems.
- Dynamic algorithms use memorization.



When to Use Dynamic Programming (DP)?

- Dynamic programming is used for solving problems that consists of the following characteristics:

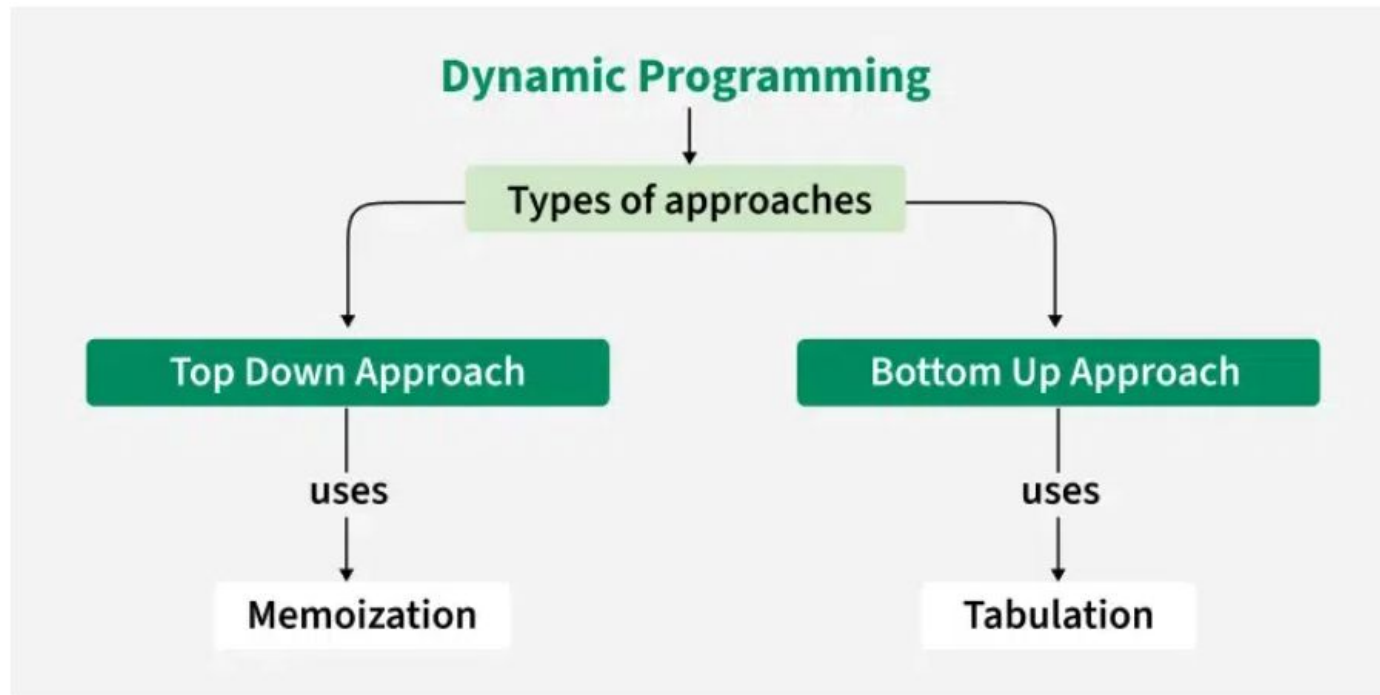
1. Optimal Substructure:

- The property [Optimal substructure](#) means that we use the optimal results of subproblems to achieve the optimal result of the bigger problem.
- **Example:**
- Consider the problem of finding the **minimum cost** path in a weighted graph from a **source** node to a **destination** node. We can break this problem down into smaller subproblems:
- Find the **minimum cost** path from the **source** node to each **intermediate** node.
- Find the **minimum cost** path from each **intermediate** node to the **destination** node.
- The solution to the larger problem (finding the minimum cost path from the source node to the destination node) can be constructed from the solutions to these smaller subproblems.

2. Overlapping Subproblems:

- The same subproblems are solved repeatedly in different parts of the problem refer to [Overlapping Subproblems Property in Dynamic Programming](#).
- **Example:**
- Consider the problem of computing the [Fibonacci series](#). To compute the Fibonacci number at index **n**, we need to compute the Fibonacci numbers at indices **n-1** and **n-2**. This means that the subproblem of computing the Fibonacci number at index **n-2** is used twice (note that the call for **n-1** will make two calls, one for **n-2** and other for **n-3**) in the solution to the larger problem of computing the Fibonacci number at index **n**.

Approaches of Dynamic Programming (DP)



Approaches of Dynamic Programming (DP)

1. Top-Down Approach (Memoization):

- In the top-down approach, also known as [memoization](#), we keep the solution recursive and add a memoization table to avoid repeated calls of same subproblems.
- Before making any recursive call, we first check if the memoization table already has solution for it.
- After the recursive call is over, we store the solution in the memoization table.

• 2. Bottom-Up Approach (Tabulation):

- In the bottom-up approach, also known as **tabulation**, we start with the **smallest subproblems** and gradually build up to the final solution.
- We write an iterative solution (avoid recursion overhead) and build the solution in bottom-up manner.
- We use a dp table where we first fill the solution for base cases and then fill the remaining entries of the table using recursive formula.
- We only use recursive formula on table entries and do not make recursive calls.

Floyd-Warshall

- The Floyd-Warshall algorithm is a graph algorithm that is deployed to find the shortest path between all the vertices present in a weighted graph.
- This algorithm is different from other shortest path algorithms; to describe it simply, this algorithm uses each vertex in the graph as a pivot to check if it provides the shortest way to travel from one point to another.
- Floyd-Warshall algorithm works on both directed and undirected weighted graphs unless these graphs do not contain any negative cycles in them. By negative cycles, it is meant that the sum of all the edges in the graph must not lead to a negative number.
- Since, the algorithm deals with overlapping sub-problems the path found by the vertices acting as pivot are stored for solving the next steps it uses the dynamic programming approach.
- Floyd-Warshall algorithm is one of the methods in All-pairs shortest path algorithms and it is solved using the Adjacency Matrix representation of graphs.

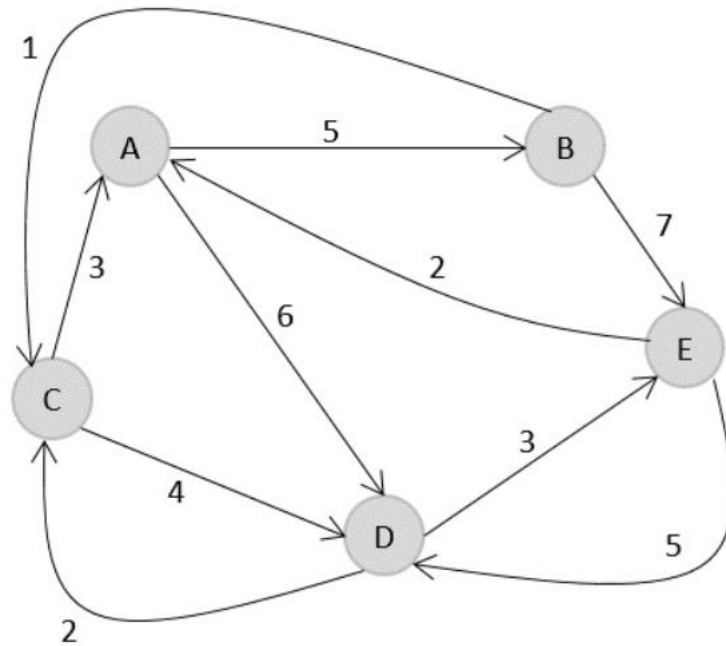
Floyd-Warshall Algorithm

- Consider a graph, $G = \{V, E\}$ where V is the set of all vertices present in the graph and E is the set of all the edges in the graph. The graph, G , is represented in the form of an adjacency matrix, A , that contains all the weights of every edge connecting two vertices.

Algorithm

- Step 1 – Construct an adjacency matrix A with all the costs of edges present in the graph. If there is no path between two vertices, mark the value as ∞ .
- Step 2 – Derive another adjacency matrix $A1$ from A keeping the first row and first column of the original adjacency matrix intact in $A1$. And for the remaining values, say $A1[i,j]$, if $A[i,j] > A[i,k] + A[k,j]$ then replace $A1[i,j]$ with $A[i,k] + A[k,j]$. Otherwise, do not change the values. Here, in this step, $k = 1$ (first vertex acting as pivot).
- Step 3 – Repeat Step 2 for all the vertices in the graph by changing the k value for every pivot vertex until the final matrix is achieved.
- Step 4 – The final adjacency matrix obtained is the final solution with all the shortest paths.

Floyd-Warshall



Floyd-Warshall

- Step 1-Construct an adjacency matrix A with all the distances as values.

$$A = \begin{matrix} & \begin{matrix} 0 & 5 & \infty & 6 & \infty \end{matrix} \\ \begin{matrix} \infty & 0 & 1 & \infty & 7 \end{matrix} & \\ \begin{matrix} 3 & \infty & 0 & 4 & \infty \end{matrix} & \\ \begin{matrix} \infty & \infty & 2 & 0 & 3 \end{matrix} & \\ \begin{matrix} 2 & \infty & \infty & 5 & 0 \end{matrix} & \end{matrix}$$

Floyd-Warshall

- Step 2- Considering the above adjacency matrix as the input, derive another matrix A_0 by keeping only first rows and columns intact. Take $k = 1$, and replace all the other values by $A[i,k] + A[k,j]$.

$$A = \begin{matrix} & 0 & 5 & \infty & 6 & \infty \\ \infty & & & & & \\ 3 & & & & & \\ \infty & & & & & \\ 2 & & & & & \end{matrix}$$

$$A_1 = \begin{matrix} & 0 & 5 & \infty & 6 & \infty \\ \infty & 0 & 1 & \infty & 7 & \\ 3 & 8 & 0 & 4 & \infty & \\ \infty & \infty & 2 & 0 & 3 & \\ 2 & 7 & \infty & 5 & 0 & \end{matrix}$$

Floyd-Warshall

- Step 3- Considering the above adjacency matrix as the input, derive another matrix A_0 by keeping only first rows and columns intact. Take $k = 1$, and replace all the other values by $A[i,k] + A[k,j]$.

$$A_2 = \begin{matrix} & & 5 & & & \\ & \infty & 0 & 1 & \infty & 7 \\ & 8 & & & & \\ & \infty & & & & \\ & 7 & & & & \\ & 0 & 5 & 6 & 6 & 12 \\ & \infty & 0 & 1 & \infty & 7 \\ A_2 = & 3 & 8 & 0 & 4 & 15 \\ & \infty & \infty & 2 & 0 & 3 \\ & 2 & 7 & 8 & 5 & 0 \end{matrix}$$

Floyd-Warshall

- Step 4- Considering the above adjacency matrix as the input, derive another matrix A_0 by keeping only first rows and columns intact. Take $k = 1$, and replace all the other values by $A[i,k] + A[k,j]$.

$$\begin{array}{c}
 6 \\
 1 \\
 A_3 = \begin{array}{ccccc}
 3 & 8 & 0 & 4 & 15 \\
 2 \\
 8 \\
 \begin{array}{ccccc}
 0 & 5 & 6 & 6 & 12 \\
 4 & 0 & 1 & 5 & 7 \\
 A_3 = \begin{array}{ccccc}
 3 & 8 & 0 & 4 & 15 \\
 5 & 10 & 2 & 0 & 3 \\
 2 & 7 & 8 & 5 & 0
 \end{array}
 \end{array}
 \end{array}$$

Floyd-Warshall

- Step 5- Considering the above adjacency matrix as the input, derive another matrix A_0 by keeping only first rows and columns intact. Take $k = 1$, and replace all the other values by $A[i,k]+A[k,j]$.

$$A_4 = \begin{matrix} & & & 6 & & \\ & & & 5 & & \\ & & & 4 & & \\ 5 & 10 & 2 & 0 & 3 & \\ & & & 5 & & \\ & 0 & 5 & 6 & 6 & 9 \\ 4 & 0 & 1 & 5 & 7 & \\ A_4 = 3 & 8 & 0 & 4 & 7 & \\ & 5 & 10 & 2 & 0 & 3 \\ & 2 & 7 & 7 & 5 & 0 \end{matrix}$$

Floyd-Warshall

- Step 6- Considering the above adjacency matrix as the input, derive another matrix A_0 by keeping only first rows and columns intact. Take $k = 1$, and replace all the other values by $A[i,k] + A[k,j]$.

$$A_5 = \begin{matrix} & & & & 9 \\ & & & & 7 \\ & & & & 7 \\ & & & & 3 \\ & 2 & 7 & 7 & 5 & 0 \\ & 0 & 5 & 6 & 6 & 9 \\ & 4 & 0 & 1 & 5 & 7 \\ A_5 = & 3 & 8 & 0 & 4 & 7 \\ & 5 & 10 & 2 & 0 & 3 \\ & 2 & 7 & 7 & 5 & 0 \end{matrix}$$

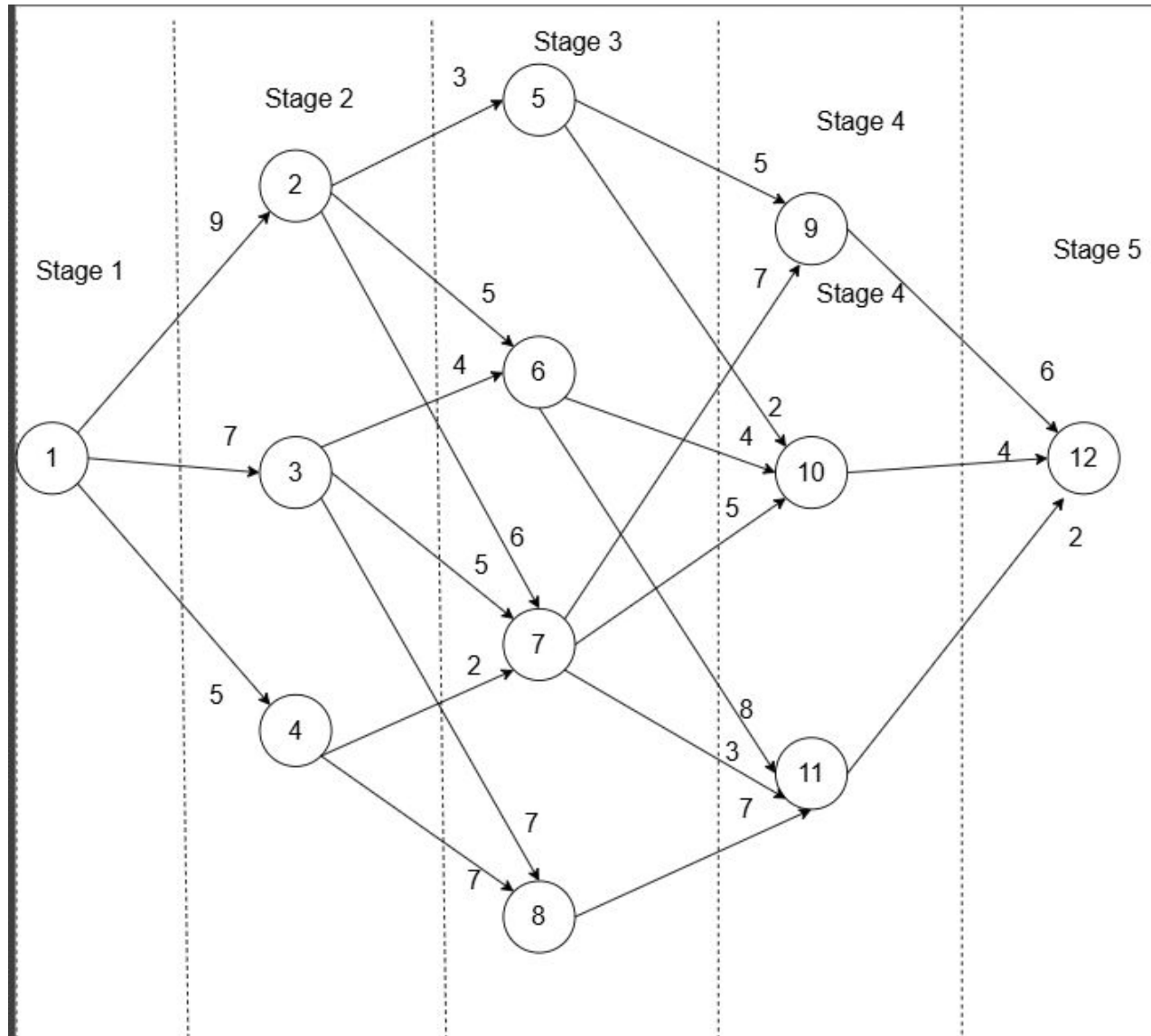
Floyd-Warshall Analysis

- From the pseudocode above, the Floyd-Warshall algorithm operates using three for loops to find the shortest distance between all pairs of vertices within a graph.
- Therefore, the time complexity of the Floyd-Warshall algorithm is $O(n^3)$, where n is the number of vertices in the graph.
- The space complexity of the algorithm is $O(n^2)$.

Multistage Graph

- A multistage graph $G=(V, E)$ is a directed and weighted graph in which vertices are divided into stages (the first stage and last stage of which will have a single vertex to represent the starting vertex or ending vertex).
- In between the starting and ending vertex, there will be vertices in different stages that connect the starting and ending vertex.
- The main aim of this graph is to find the minimum cost path between starting and ending vertex.

Multistage Graph



Multistage Graph

- In the above graph, cost of an edge is represented as $c(i, j)$. We need to find the minimum cost path from vertex 1 to vertex 12. Using the below formula we can find the shortest cost path from source to destination:

$$cost(i, j) = \min_l c(j, l) + cost(i+1, l)$$

Multistage Graph

- Step 1
- Step 1 uses the forwarded approach ($\text{cost}(5,12) = 0$).
- Here, 5 represents the stage number and 12 represents a node in that stage. Since there are no outgoing edges from vertex 12, the cost is 0.
- Step 2
- $\text{cost}(4,9)=c(9,12)=6$
- $\text{cost}(4,10)=c(10,12)=4$ $\text{cost}(4,11)=c(11,12)= 2$

Multistage Graph

- Step 3
- $\text{cost}(3,5) = \min\{c(5,9) + \text{cost}(4,9), c(5,10) + \text{cost}(4,10)\}$
- $\min\{5+6, 2+4\}$
- $\min\{11, 6\} = 6$
- $\text{cost}(3,5) = 6$

- $\text{cost}(3,6) = \min\{c(6,10) + \text{cost}(4,10), c(6,11) + \text{cost}(4,11)\}$
- $\min\{4+4, 8+2\}$
- $\min\{8, 10\} = 8$
- $\text{cost}(3,6) = 8$

- $\text{cost}(3,7) = \min\{c(7,9) + \text{cost}(4,9), c(7,10) + \text{cost}(4,10), c(7,11) + \text{cost}(4,11)\}$
 $\min\{7+6, 5+4, 3+2\}$
- $\min\{13, 9, 5\} = 5$
- $\text{cost}(3,7) = 5$

Multistage Graph

- Step 4
- $\text{cost}(2,2) = \min\{c(2,5) + \text{cost}(3,5), c(2,6) + \text{cost}(3,6), c(2,7) + \text{cost}(3,7)\}$
 $\min\{3+6, 5+8, 6+5\}$
- $\min\{9, 13, 11\} = 9$
- $\text{cost}(2,2) = 9$

- $\text{cost}(2,3) = \min\{c(3,6) + \text{cost}(3,6), c(3,7) + \text{cost}(3,7), c(3,8) + \text{cost}(3,8)\}$
 $\min\{4+8, 5+5, 7+9\}$
- $\min\{12, 10, 16\} = 10$
- $\text{cost}(2,3) = 10$

- $\text{cost}(2,4) = \min\{c(4,7) + \text{cost}(3,7), c(4,8) + \text{cost}(3,8)\}$
- $\min\{2+5, 7+9\}$
- $\min\{7, 16\} = 7$
- $\text{cost}(2,4) = 7$

Multistage Graph

- Step-5
- $\text{cost}(1,1) = \min\{c(1,2) + \text{cost}(2,2), c(1,3) + \text{cost}(2,3), c(1,4) + \text{cost}(2,4)\}$
 $\min\{9+9, 7+10, 5+7\}$
- $\min\{18, 17, 12\} = 12$
- $\text{cost}(1,1) = 12$
- From the above calculations we get:

v	1	2	3	4	5	6	7	8	9	10	11	12
cost	12	9	10	7	6	8	5	9	6	4	2	0
d	4	5	7	7	10	10	11	11	12	12	12	12

Here, d represents which vertex has given minimum cost to above vertex

Multistage Graph

Therefore:

```
d[1]=4  
d[4]=7  
d[7]=11  
d[11]=12  
shortest path 1--4--7--11--12 i.e 12
```